

ALL DESIGN PATTERNS

IN ONE PLACE

Creational

- Singleton
- Factory Method
- Abstract Factory
- Builder
- Prototype

Structural

- Adapter
- Decorator
- Facade
- Proxy
- Composite

Behavioral

- Observer
- Strategy
- Command
- State
- Iterator



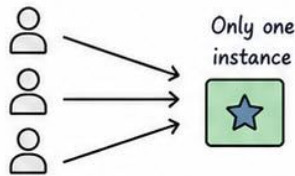
Swipe to explore

Creational Design Patterns

Creational Design Patterns are design patterns that deal with object creation mechanisms, making systems more flexible, reusable, and maintainable.

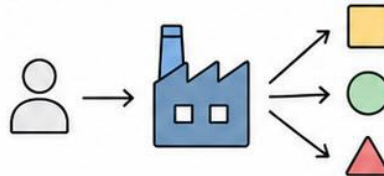
Quick Visual Guide

1 Singleton



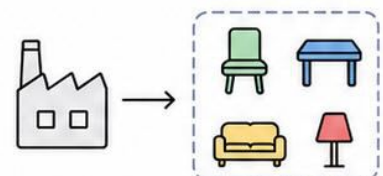
Ensures a class has only one instance and provides a global point of access to it.

2 Factory Method



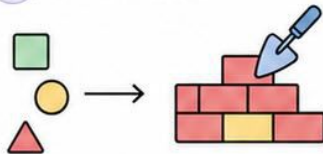
Defines an interface for creating an object, but lets subclasses decide which class to instantiate.

3 Abstract Factory



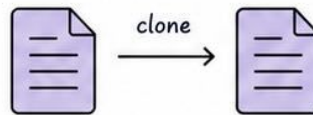
Provides an interface for creating families of related or dependent objects without specifying their concrete classes.

4 Builder



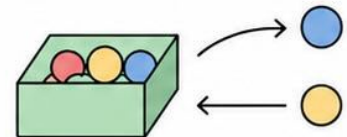
Separates the construction of a complex object from its representation so that the same construction process can create different representations.

5 Prototype



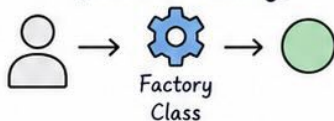
Creates new objects by copying an existing object (prototype) instead of creating from scratch.

6 Object Pool



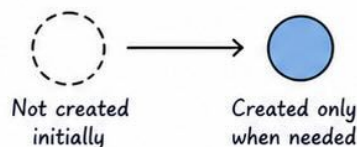
Reuses existing objects instead of creating new ones, improving performance and resource usage.

7 Factory Method (Static Factory)



A static method in a class that returns an instance of an object. (Alternative to constructor)

8 Lazy Initialization



Delays the creation of an object until it is actually required.

9 Dependency Injection



An object receives its dependencies from external sources rather than creating them itself.

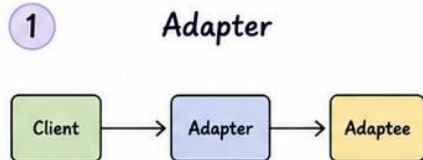


Comment notes for detailed notes of each pattern

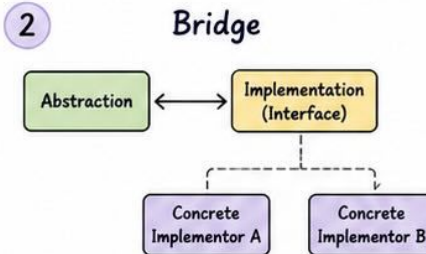
≡ Structural Design Patterns ≡

Structural Design Patterns focus on how classes and objects are composed to form larger structures and how these structures can be made more flexible and efficient.

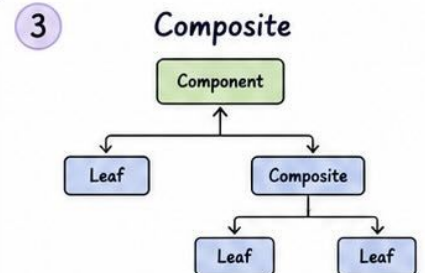
Quick Visual Guide



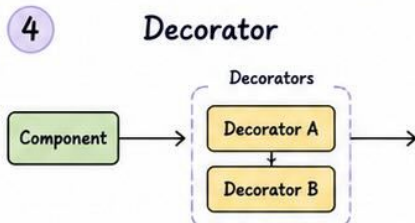
Allows incompatible interfaces to work together by acting as a bridge between them.



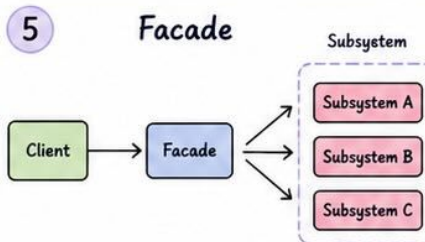
Decouples an abstraction from its implementation so that both can vary independently.



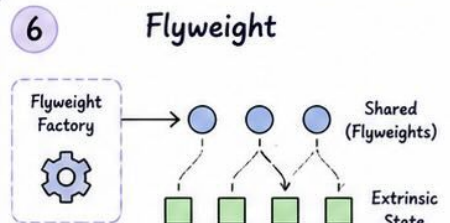
Composes objects into tree structures to represent part-whole hierarchies, treating individual objects and compositions uniformly.



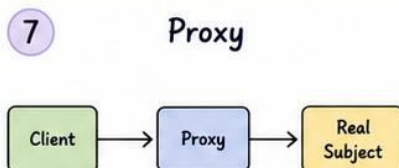
Adds additional responsibilities to an object dynamically without altering its structure.



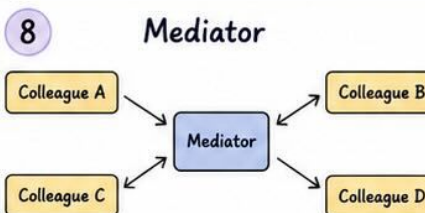
Provides a simplified interface to a complex subsystem, reducing the learning curve and dependencies.



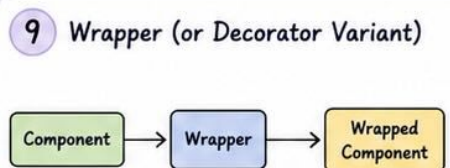
Shares common parts of state among multiple objects to minimize memory usage.



Provides a surrogate or placeholder to control access to another object.



Defines an object that encapsulates how a set of objects interact, promoting loose coupling.



Encapsulates an object to provide additional behavior or interface variations.

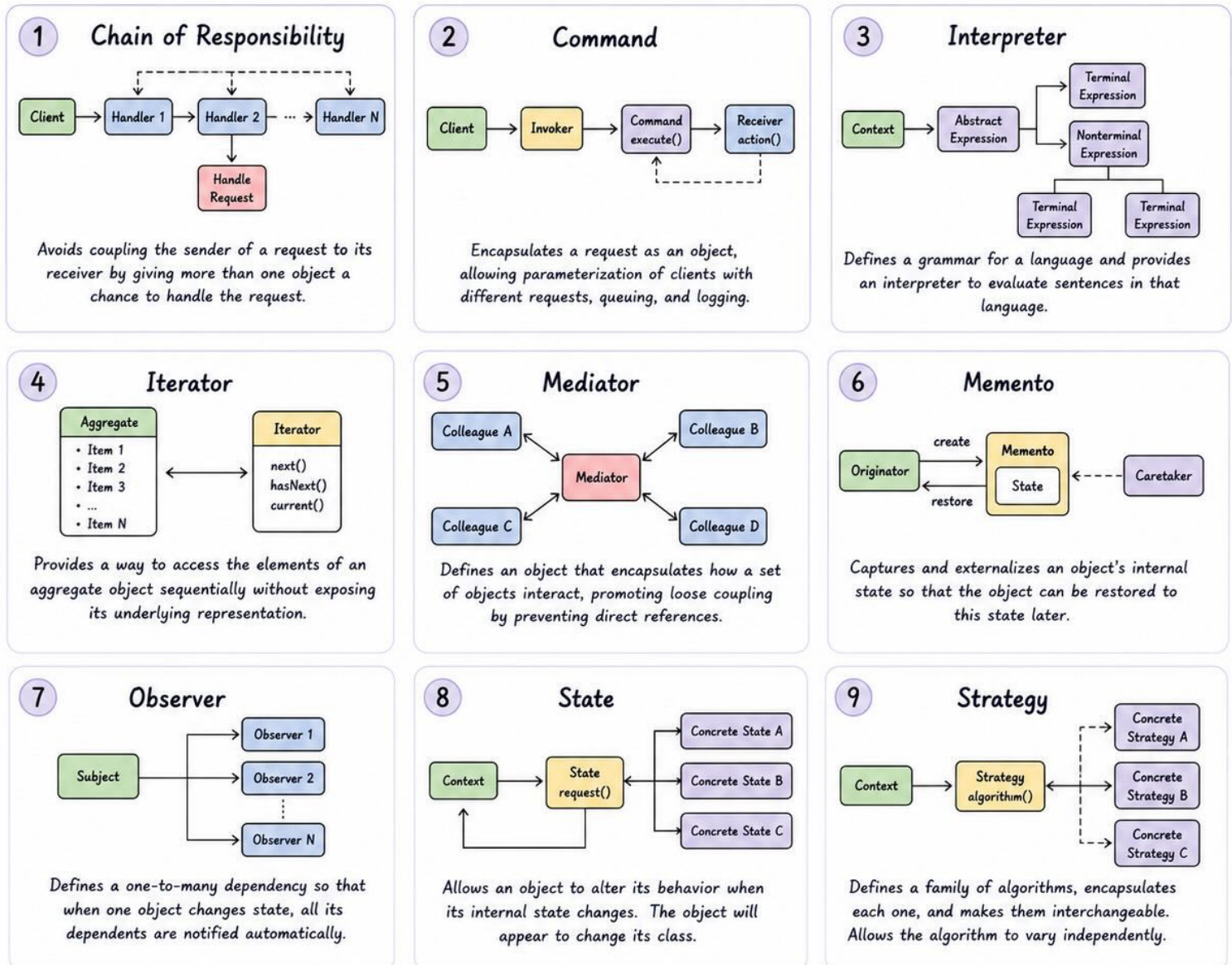


Comment notes for detailed notes of each pattern

Behavioural Design Patterns

Behavioural Design Patterns focus on communication between objects, how they interact, distribute responsibilities, and manage algorithms and control flow.

Quick Visual Guide



Comment notes for detailed notes of each pattern